# A view of 0 or 1 elements: `view::maybe`


- Document number: P1255R4
- Date: 2019-06-17


- Author: Steve Downey <sdowney2@bloomberg.net>, <sdowney@gmail.com>

- Audience: LEWG


Abstract: This paper proposes `view::maybe` a range adaptor that produces a view with cardinality 0 or 1 which adapts nullable types such as `std::optional` and pointer to object types.


## Table of Contents



# 1 Changes

## 1.1 Changes since R3

### 1.1.1 Always Capture

### 1.1.2 Support reference_wrapper

## 1.2 Changes since R2

### 1.2.1 Reflects current code as reviewed

### 1.2.2 Nullable concept specification

Remove Readable as part of the specification, use the useful requirements from Readable

### 1.2.3 Wording for view::maybe as proposed

### 1.2.4 Appendix A: wording for a view_maybe that always captures

## 1.3 Changes since R1

### 1.3.1 Refer to view::all

Behavior of capture vs refer is similar to how view::all works over the expression it is given

### 1.3.2 Use wording 'range adaptor object'

Match current working paper language

## 1.4 Changes since R0

### 1.4.1 Remove customization point objects

Removed view::maybe_has_value and view::maybe_value, instead requiring that the nullable type be dereferenceable and contextually convertible to bool.

### 1.4.2 Concept `Nullable`, for exposition

Concept Nullable, which is Readable and contextually convertible to bool

### 1.4.3 Capture rvalues by decay copy

Hold a copy when constructing a view over a nullable rvalue.

### 1.4.4 Remove maybe_view as a specified type

Introduced two exposition types, one safely holding a copy, the other referring to the nullable

# 2 Motivation

In writing range transformation pipelines it is useful to be able to lift a nullable value into a view that is either empty or contains the value held by the nullable. The adapter `view::single` fills a similar purpose for non-nullable values, lifting a single value into a view, and `view::empty` provides a range of no values of a given type. A `view::maybe` adaptor also allows nullable values to be treated as ranges when it is otherwise undesirable to make them containers, for example `std::optional`.

```
std::vector<std::optional<int>> v{
  std::optional<int>{42},
  std::optional<int>{},
  std::optional<int>{6 * 9}};

auto r = view::join(view::transform(v, view::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 54
}
```

In addition to range transformation pipelines, `view::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an if statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the `if` is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK ?
    }
```

```
}

for (auto&& opt : view::maybe(possible_value())) {
    // a few dozen lines ...
    use(opt); // opt is OK
}
```

The view can be on a `std::reference_wrapper`, allowing the underlying nullable to be modified:

```
std::optional o{7};
for (auto&& i : view::maybe(std::ref(o))) {
    i = 9;
    std::cout << "i=" << i << " prints 9\n";
}
std::cout << "o=" << *o << " prints 9\n";
```

Of course, if the nullable is empty, there is nothing in the view to modify.

```
auto oe = std::optional<int>{};
for (int i : view::maybe(std::ref(oe)))
    std::cout << "i=" << i << '\n'; // does not print
```

Converting an optional type into a view can make APIs that return optional types, such a lookup operations, easier to work with in range pipelines.

```
std::unordered_set<int> set{1, 3, 7, 9};

auto flt = [=](int i) -> std::optional<int> {
    if (set.contains(i))
        return i;
    else
        return {};
};

for (auto i : ranges::iota_view{1, 10} | ranges::view::transform(flt)) {
    for (auto j : view::maybe(i)) {
        for (auto k : ranges::iota_view(0, j))
            std::cout << '\a';
        std::cout << '\n';
    }
}

// Produce 1 ring, 3 rings, 7 rings, and 9 rings
```

# 3 Proposal

Add a range adaptor object `view::maybe`, returning a view over a nullable object, capturing by value temporary nullables. A `Nullable` object is one that is both contextually convertible to bool and for which the type produced by dereferencing is an equality preserving object. Non void pointers, `std::optional`, and the proposed `outcome` and `expected` types all model Nullable. Function pointers do not, as functions are not objects. Iterators do not generally model Nullable, as they are not required to be contextually convertible to bool.

# 4 Design

The basis of the design is to hybridize `view::single` and `view::empty`. If the underlying object claims to hold a value, as determined by checking if the object when converted to bool is true, `begin` and `end` of the view are equivalent to the address of the held value within the underlying object and one past the underlying object. If the underlying object does not have a value, `begin` and `end` return `nullptr`.

# 5 Synopsis

## 5.1 Maybe View

`view::maybe` returns a View over a Nullable that is either empty if the nullable is empty, or provides access to the contents of the nullable object.

The name view::maybe denotes a range adaptor object ([range.adaptor.object]). For some subexpression E, the expression view::maybe(E) is expression-equivalent to:

– maybe_view{E}, the View specified below, if the expression is well formed, where decay-copy(E) is moved into the maybe_view

– otherwise view::maybe(E) is ill-formed.

[Note: Whenever view::maybe(E) is a valid expression, it is a prvalue whose type models View. — end note ]

## 5.2 Concept *Nullable*

Types that:

– are contextually convertible to bool

– are dereferenceable

– have const references which are dereferenceable

– the iter_reference_t of the type and the iter_reference_t of the const type, will :

– satisfy is_lvalue_reference

– satisfy if_object when the reference is removed

– for const pointers to the referred to types, satisfy ConvertibleTo model the exposition only Nullable concept

– Or are a reference_wrapper around a type that satifies Nullable

Given a value i of type I, I models Nullable only if the expression *i is equality-preserving. [ Note: The expression *i is indirectly required to be valid via the exposition-only dereferenceable concept ([iterator.synopsis]). — end note ]

```
namespace std::ranges {

// For Exposition
template <class T, class Ref, class ConstRef>
concept bool _ReadableReferences =
    is_lvalue_reference_v<Ref> &&
    is_object_v<remove_reference_t<Ref>> &&
    is_lvalue_reference_v<ConstRef> &&
    is_object_v<remove_reference_t<ConstRef>> &&
```

```
    ConvertibleTo<add_pointer_t<ConstRef>,
                  const remove_reference_t<Ref>*>;

template <class T>
concept bool Nullable =
    is_object_v<T> &&
    requires(T& t, const T& ct) {
        bool(ct); // Contextually bool
        *t; // T& is deferenceable
        *ct; // const T& is deferenceable
    }
    && _ReadableReferences<T,
                           iter_reference_t<T>,        // Ref
                           iter_reference_t<const T>>; // ConstRef

template <class T>
concept bool WrappedNullable =
is_reference_wrapper<T> && Nullable<typename T::type>;
```

## 5.3 *maybe_view*

```
template <typename Maybe>
requires ranges::CopyConstructible<Maybe> &&
(Nullable<Maybe> ||
 WrappedNullable<Maybe>)
class maybe_view
    : public ranges::view_interface<maybe_view<Maybe>> {
  private:
// For Exposition
    using T = /* see below */
    semiregularbox<Maybe> value_;

  public:
    constexpr maybe_view() = default;
    constexpr explicit maybe_view(Maybe const& maybe)
        noexcept(std::is_nothrow_copy_constructible_v<Maybe>);

    constexpr explicit maybe_view(Maybe&& maybe)
        noexcept(std::is_nothrow_move_constructible_v<Maybe>);

    template<class... Args>
    requires Constructible<Maybe, Args...>
    constexpr maybe_view(in_place_t, Args&&... args);

    constexpr T*       begin() noexcept;
    constexpr const T* begin() const noexcept;
    constexpr T*       end() noexcept;
    constexpr const T* end() const noexcept;

    constexpr std::ptrdiff_t size() const noexcept;

    constexpr T* data() noexcept;
    constexpr const T* data() const noexcept;
};

constexpr explicit maybe_view(const Maybe& maybe);
}
```

Where the type alias T is the iter_reference_t with the reference removed of either the type Maybe or the type reference_wrapper<Maybe>::type.

```
// For Exposition
template <typename T>
struct maybe_unwrap {
    using type = T;
};

template <class T>
struct maybe_unwrap<std::reference_wrapper<T>> {
    using type = T;
};

using T = std::remove_reference_t<
    ranges::iter_reference_t<typename maybe_unwrap<Maybe>::type>>;
```

```
constexpr explicit maybe_view(Maybe const& maybe)
    noexcept(std::is_nothrow_copy_constructible_v<Maybe>);
```

*Effects*: Initializes value_ with maybe. 🔗

```
constexpr explicit maybe_view(Maybe&& maybe)
    noexcept(std::is_nothrow_move_constructible_v<Maybe>);
```

*Effects*: Initializes value_ with `std::move(maybe)`. 🔗

```
template<class... Args>
constexpr maybe_view(in_place_t, Args&&... args);
```

*Effects*: Initializes value_ as if by `value_{in_place, std::forward<Args>(args)...}`. 🔗

```
constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;
```

*Effects*: Equivalent to: `return data();`. 🔗

```
constexpr T* end() noexcept;
constexpr const T* end() const noexcept;
```

*Effects*: Equivalent to: `return data() + size();`. 🔗

```
static constexpr ptrdiff_t size() noexcept;
```

*Effects*: Equivalent to:

```
        if constexpr (is_reference_wrapper<Maybe>) {
            return bool(value_.get().get());
        } else {
            return bool(value_.get());
        }
```

🔗

```
constexpr T* data() noexcept;
```

*Effects*: Equivalent to:

```
        Maybe& m = value_.get();
        if constexpr (is_reference_wrapper<Maybe>) {
            return m.get() ? std::addressof(*(m.get())) : nullptr;
        } else {
            return m ? std::addressof(*m) : nullptr;
        }
```

```
constexpr const T* data() const noexcept;
```

*Effects*: Equivalent to:

```
        const Maybe& m = value_.get();
        if constexpr (is_reference_wrapper<Maybe>) {
            return m.get() ? std::addressof(*(m.get())) : nullptr;
        } else {
            return m ? std::addressof(*m) : nullptr;
        }
```

# 6 Impact on the standard

A pure library extension, affecting no other parts of the library or language.

# 7 References

[P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella. The One Ranges Proposal URL: https://wg21.link/p0896r3

[P0323R7] Vicente Botet, JF Bastien. std::expected URL: https://wg21.link/p0323r7